

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия, Системное и
прикладное программное обеспечение

Дисциплина: Основы профессиональной деятельности

Лабораторная работа №5
Асинхронный обмен данными с ВУ
Вариант 9533

Выполнил:
Карнажицкий Максим Романович
Группа: Р3111

Проверила:
Остапенко Ольга Дмитриевна

г. Санкт-Петербург, 2025

Оглавление

Описание задания.....	3
Текст исходной программы.....	4
Описание программы.....	6
Область представления.....	7
Область допустимых значений.....	8
Размещение программы.....	8
Трассировка программы.....	9
Вывод.....	12

Описание задания

По выданному преподавателем варианту разработать программу асинхронного обмена данными с внешним устройством. При помощи программы осуществить ввод или вывод информации, используя в качестве подтверждения данных сигнал (кнопку) готовности ВУ.

Введите номер варианта

1. Программа осуществляет асинхронный ввод данных с ВУ-2
2. Программа начинается с адреса $2CF_{16}$. Размещаемая строка находится по адресу $59F_{16}$.
3. Строка должна быть представлена в кодировке ISO-8859-5.
4. Формат представления строки в памяти: АДР1: СИМВ1 СИМВ2 АДР2: СИМВ3 СИМВ4 ... СТОП_СИМВ.
5. Ввод или вывод строки должен быть завершен по символу с кодом 0A (NL). Стоп символ является обычным символом строки и подчиняется тем же правилам расположения в памяти что и другие символы строки.

Рисунок 1. Вариант 9533

Текст исходной программы

Таблица 1. Текст программы варианта №8823

Адрес	Содержимое		Комментарии
	Код команды	Мнемоника	
2CF	059F	RES_ADDR	адрес первой ячейки итоговой строки
2D0	0000	CURR_POINTER	текущий (последний) символ
2D1	0040	IS_READY	константа для определения готовности ВУ-2
2D2	000A	EOF	символ окончания ввода
2D3	00FF	PART_CHECK	маска для взятия младшего байта
2D4	0200	CLA	текущий элемент - первый элемент списка
2D5	AEF8	LD RES_ADDR	
2D6	EEF8	ST	
2D7	0200	<i>S1</i> : CLA	ожидание ввода (spin-loop)
2D8	1205	IN 5	
2D9	2EF6	AND IS_READY	
2DA	F0FC	BEQ S1	
2DB	1204	IN 4	Ввод старшего байта и помещение для хранения в нужную ячейку
2DC	0680	SWAB	
2DD	E8F1	ST(CURR_POINTER)	
2DE	0680	SWAB	Сравнение с EOF. Если это конец ввода, то записанный символ уже сохранен и программа завершает работу. Иначе переходим в S2
2DF	7EF1	CMP EOF	
2E0	F101	BNE S2	
2E1	CE0A	JUMP BREAK	
2E2	1205	<i>S2</i> : IN 5	ожидание ввода (spin-loop)
2E3	2EEC	AND IS_READY	
2E4	F0FD	BEQ S2	
2E5	AE8A	LD (CURR_POINTER)	Совмещение с предыдущим символом в одну пару и дальнейшее сохранение в "charset"
2E6	1204	IN 4	

2E7	EAE8	ST (CURR_POINTER)+	
2E8	2EE9	AND PART_CHECK	Проверка EOF. Если младший байт EOF, то завершение работы. Иначе возвращаемся на S1.
2E9	7EE7	CMP EOF	
2EA	F1EC	BNE S1	
2EB	CE00	JUMP BREAK	
2EC	0100	<i>BREAK</i> : HLT	Останов

Описание программы

Программа осуществляет посимвольное чтение строк из ВУ-2, используя кодировку ISO-8859-5. После того, как символ был введен впервые, он помещается в первую ячейку и хранится в старшем байте ячейки. Когда вводится второй символ, он помещается в младший байт этой же ячейки, а последующая пара символов вводится уже в новую ячейку. Если же после нечетного по счету символа ввод завершен, то программа так и сохраняет его в старшем байте (что-то типа Big-Endian).

Когда вводится 0x0A (EOF), ввод завершается, предварительно записав этот символ в память.

Исходный код:

```
1 ; Чтение из ВУ-2
2
3 ORG          0x2CF
4
5 RES_ADDR:    WORD 0x59F      ; указатель на первый символ итоговой строки
6 CURR_POINTER: WORD ?        ; текущий (последний) символ
7 IS_READY:    WORD 0x40      ; 6th bit = 1
8 EOF:         WORD 0x000A    ; eof
9 PART_CHECK:  WORD 0x00FF    ; обрезание по последнему байту
10
11 START:      CLA
12             LD      RES_ADDR
13             ST      CURR_POINTER
14
15 S1:         CLA                ; очистка после конца двух байтов
16             IN      5
17             AND     IS_READY
18             BEQ     S1         ; spin-loop
19
20             IN      4
21             SWAB                ; char -> AC[15..8]
22             ST      (CURR_POINTER) ; для облегчения работы с младшим символом
23
24             ; проверка eof
25             SWAB
26             CMP     EOF
27             BNE     S2         ; если AC != EOF, то идем в S2
28             JUMP    BREAK      ; иначе стоп
29
30
31 S2:         IN      5
32             AND     IS_READY
33             BEQ     S2         ; spin-loop
34
35             LD      (CURR_POINTER)
36             IN      4
37             ST      (CURR_POINTER)+
38
39             ; проверка EOF
40             AND     PART_CHECK
41             CMP     EOF
42             BNE     S1         ; если последнее слово НЕ eof, то продолжаем ввод в s1
43             JUMP    BREAK      ; иначе стоп
44
45 BREAK:      HLT
46
```

Область представления

RES_ADDR, CURR_POINTER – беззнаковое целое 11-разрядное число (адреса ячеек)

IS_READY, PART_CHECK – беззнаковое 16-разрядное число (константа)

EOF – беззнаковое 8-разрядное число

Область допустимых значений

$RES_ADDR, CURR_POINTER \in [2ED; 7FF - X // 2]$, где $X // 2$ – количество вводимых символов целочисленно деленное на 2
 $EOF \in [00; FF]$

Размещение программы

- Основная программа располагается на ячейках $[2CF; 2EC]$
- Массив пар символов начинается с ячейки 59F

Трассировка программы

Слово: BCOMP

	ISO-8859-5		UTF-8	UTF-16
Char	Hex code	Binary code	Hex code	Hex code
B	42	0100 0010	42	42
C	43	0100 0011	43	43
O	4F	0100 1111	4F	4F
M	4D	0100 1101	4D	4D
P	50	0101 0000	50	50
EOF	0A	0000 1010	0A	0A

Таблица 2 – Таблица трассировки

Выполняемая команда		Содержимое регистров процессора								Ячейка, содержимое которой изменилось	
Адрес	Код	IP	CR	AR	DR	SP	BR	AC	NZVC	Адрес	Новый код
2D4	0200	2D5	0200	2D4	0200	000	02D4	0000	0100		
2D5	AEF9	2D6	AEF9	2CF	059F	000	FFF9	059F	0000		
2D6	EEF9	2D7	EEF9	2D0	059F	000	FFF9	059F	0000	2D0	059F
2D7	0200	2D8	0200	2D7	0200	000	02D7	0000	0100		
2D8	1205	2D9	1205	2D8	1205	000	02D8	0040	0100		
2D9	2EF7	2DA	2EF7	2D1	0040	000	FFF7	0040	0000		
2D4	0200	2D4	0000	000	0000	000	0000	0000	0100		
2D4	0200	2D5	0200	2D4	0200	000	02D4	0000	0100		
2D5	AEF9	2D6	AEF9	2CF	059F	000	FFF9	059F	0000		
2D6	EEF9	2D7	EEF9	2D0	059F	000	FFF9	059F	0000	2D0	059F
2D7	0200	2D8	0200	2D7	0200	000	02D7	0000	0100		
2D8	1205	2D9	1205	2D8	1205	000	02D8	0040	0100		
2D9	2EF7	2DA	2EF7	2D1	0040	000	FFF7	0040	0000		
2E2	1205	2E3	1205	2E2	1205	000	02E2	0040	0001		
2E3	2EED	2E4	2EED	2D1	0040	000	FFED	0040	0001		
2E4	F0FD	2E5	F0FD	2E4	F0FD	000	02E4	0040	0001		
2E5	A8EA	2E6	A8EA	59F	4200	000	FFEA	4200	0001		
2E6	1204	2E7	1204	2E6	1204	000	02E6	4243	0001		
2E7	EAE8	2E8	EAE8	59F	4243	000	FFE8	4243	0001	2D0	05A0
										59F	4243
2E8	2EEA	2E9	2EEA	2D3	00FF	000	FFEA	0043	0001		
2E9	7EE8	2EA	7EE8	2D2	000A	000	FFE8	0043	0001		
2EA	F1EC	2D7	F1EC	2EA	F1EC	000	FFEC	0043	0001		
2D7	0200	2D8	0200	2D7	0200	000	02D7	0000	0101		
2D8	1205	2D9	1205	2D8	1205	000	02D8	0040	0101		
2D9	2EF7	2DA	2EF7	2D1	0040	000	FFF7	0040	0001		
2DA	F0FC	2DB	F0FC	2DA	F0FC	000	02DA	0040	0001		
2DB	1204	2DC	1204	2DB	1204	000	02DB	000A	0001		
2DC	0680	2DD	0680	2DC	0680	000	02DC	0A00	0001		

2DD	E8F2	2DE	E8F2	5A0	0A00	000	FFF2	0A00	0001	5A0	0A00
2DE	0680	2DF	0680	2DE	0680	000	02DE	000A	0001		
2DF	7EF2	2E0	7EF2	2D2	000A	000	FFF2	000A	0101		
2E0	F101	2E1	F101	2E0	F101	000	02E0	000A	0101		
2E1	CE0A	2EC	CE0A	2E1	02EC	000	000A	000A	0101		
2EC	0100	2ED	0100	2EC	0100	000	02EC	000A	0101		

Вывод

В ходе лабораторной работы я узнал о том, как работать с внешними устройствами в БЭВМ в асинхронном режиме, и об их устройстве. Написал полноценную программу на Ассемблере БЭВМ, а также узнал о различных кодировках.